

INF01151 – Sistemas Operacionais II

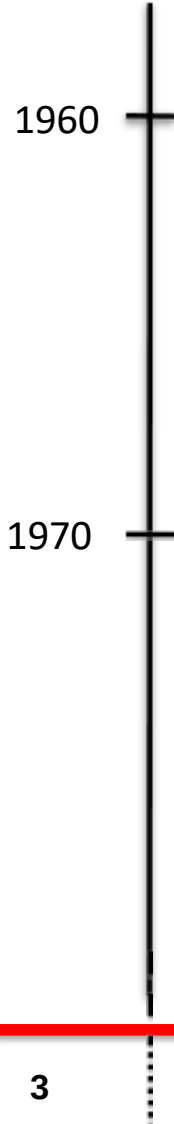
Aula 28 - Virtualização

Prof. Alberto Egon Schaeffer Filho

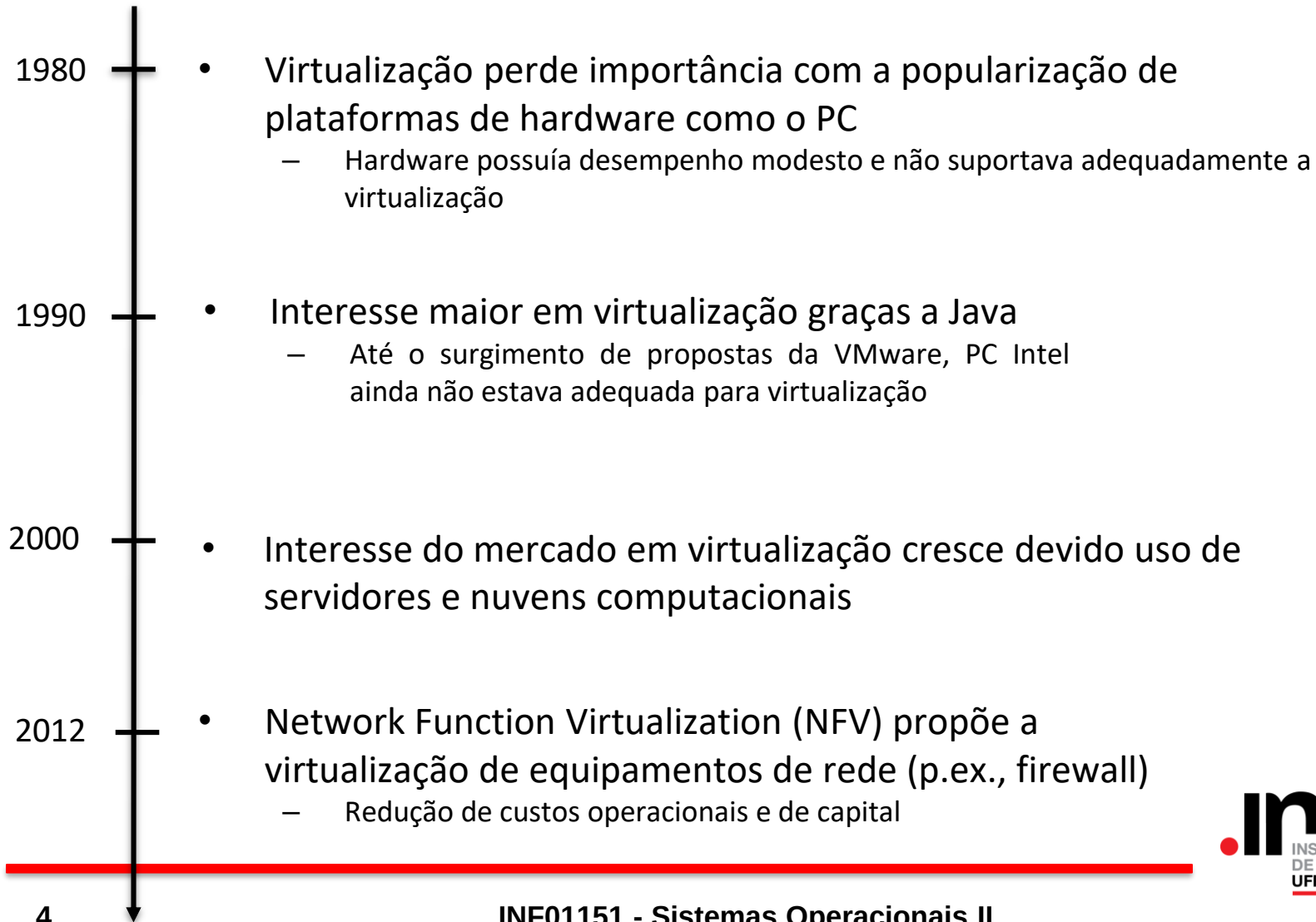
Conceito de virtualização

- Definição [Popek and Goldberg]:
 - *“Uma máquina virtual é vista como uma duplicata eficiente e isolada de uma máquina real. Essa abstração é construída por um monitor de máquina virtual (VMM - Virtual Machine Monitor).”*
- Conceito de virtualização de recursos é relativamente antigo
 - Recentes tecnologias de virtualização vem ganhando ênfase por darem suporte a importantes paradigmas de computação
 - Em ambientes de Cloud Computing, recursos podem ser adquiridos e alocados segundo a demanda
 - Consolidação de servidores
 - Elasticidade de recursos
 - Infrastructure as a Service (IaaS), Platform as a Service (PaaS), ...

Histórico

- 
- A vertical timeline line with horizontal tick marks. The years 1960 and 1970 are positioned to the left of the line. To the right of the line, bullet points describe events corresponding to these years. A thick red horizontal line spans the width of the slide near the bottom.
- 1960
 - Desenvolvimento do sistema operacional experimental M44/44X pela IBM
 - Objetivo era fornecer a cada usuário ambiente monousuário completo, com SO próprio e aplicações isoladas dos ambientes dos demais usuários
 - Início de uma série de sistemas suportando virtualização
 - 1970
 - Popek & Goldberg formalizam vários conceitos para máquinas virtuais e requisitos para que uma plataforma de hardware suporte virtualização
 - Surgimento das primeiras experiências com a execução de aplicações sobre máquinas virtuais
 - Ambiente *UCSD p-System*, no qual programas Pascal eram compilados para execução sobre o hardware virtual *P-Machine*

Histórico

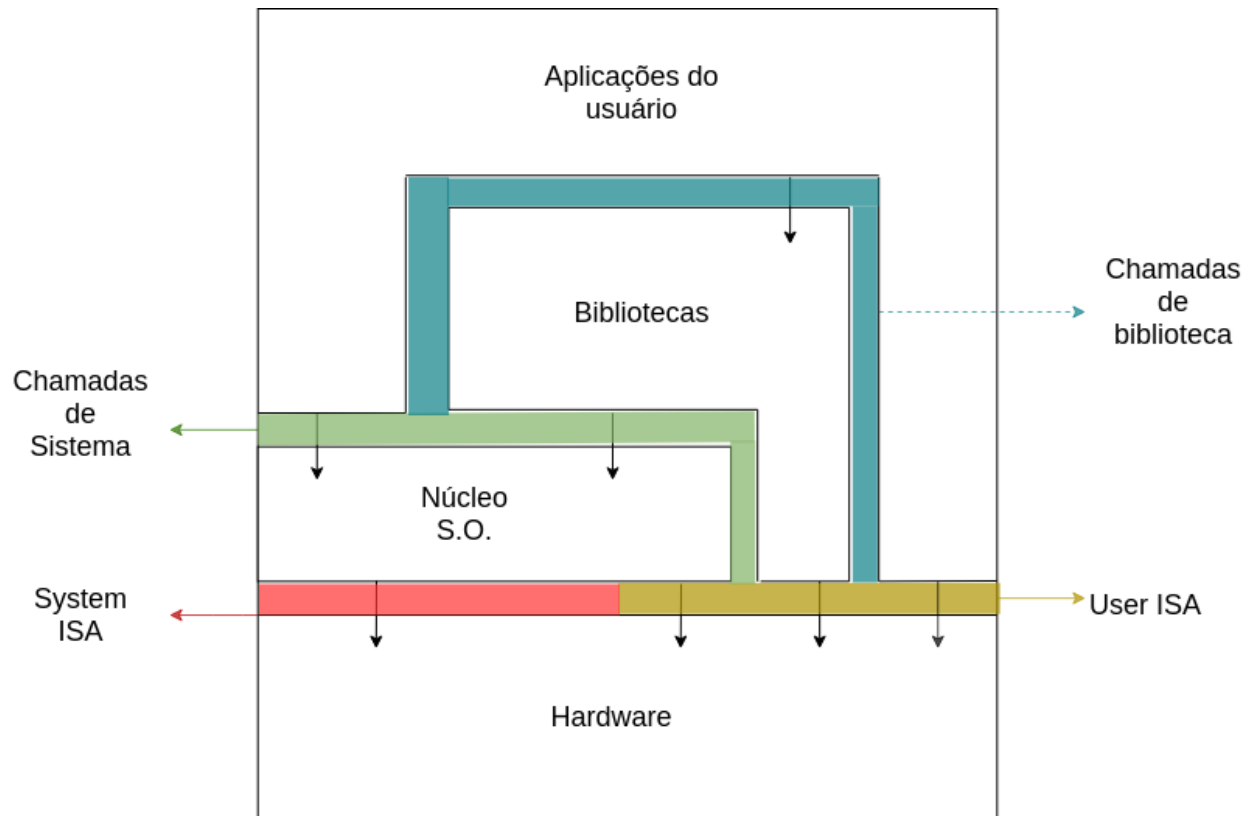


Conceito de virtualização

- Interfaces de sistema
 - Sistemas são compostos de vários componentes
 - Hardware, sistema operacional e aplicações
 - Tais componentes fornecem abstrações, que são separadas através de interfaces
 - Abstração encapsula abstrações de níveis inferiores, simplificando construção do sistemas de computação
 - Hardware fornece um conjunto de instruções para o SO
 - SO fornece conjunto de chamadas de sistema, controlando o acesso ao hardware (acesso I/O)
 - Conjunto de interfaces entre esses componentes:
 - **Conjunto de instruções (ISA)**
 - **Chamadas de sistema (syscalls)**
 - **Chamadas de biblioteca (libcalls)**

Conceito de virtualização

- Interfaces de sistema



Adaptação de Maziero, 2019

Conceito de virtualização

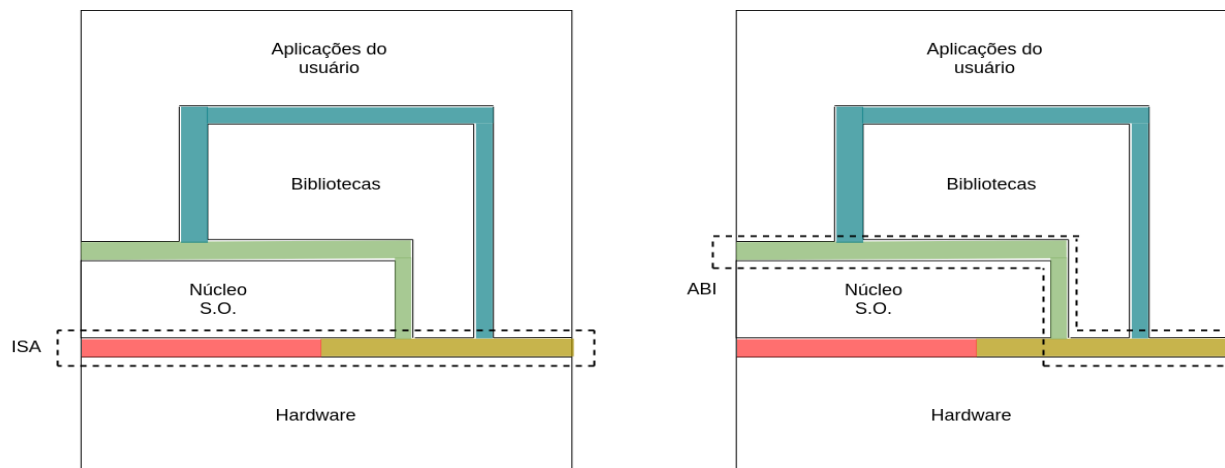
- Interfaces de sistema
 - Conjunto de instruções ISA (*Instruction Set Architecture*)
 - Interface básica entre hardware e software
 - Instruções de máquina aceitas pelo processador e operações de acesso aos recursos de hardware
 - Divididas em instruções de usuário e instruções de sistema
 - **Instruções de usuário (User ISA)**: instruções de acesso a recursos executadas em modo usuário
 - **Instruções de sistema (System ISA)**: instruções de acesso a recursos, acessíveis apenas ao núcleo do sistema, executando em modo privilegiado

Conceito de virtualização

- Interfaces de sistema
 - Chamadas de sistema (*syscalls*)
 - Conjunto de operações oferecidas pelo núcleo do sistema operacional aos processos no espaço de usuário
 - Permitem o acesso controlado das aplicações aos periféricos, à memória e às instruções privilegiadas do processador
 - Chamadas de bibliotecas (*libcalls*)
 - Bibliotecas fornecem funções para construção de programas
 - Muitas encapsulam chamadas do SO, para tornar seu uso mais simples
 - Cada biblioteca possui uma interface própria, chamada API
 - Exemplos: LibC (UNIX) com funções como `fopen()` e `printf()`

Compatibilidade entre interfaces

- Execução de programas e bibliotecas sobre uma plataforma
 - Necessário respeitar conjunto de instruções do processador em modo usuário (User ISA) e as chamadas de sistema do SO
 - Conjunto dessas duas interfaces (**User ISA + syscalls**) é chamada **Interface Binária de Aplicação (ABI – Application Binary Interface)**
 - Similarmente, SO só irá executar se tiver sido construído e compilado respeitando interface ISA (User/System ISA) do hardware



Adaptação de Maziero, 2019

Compatibilidade entre interfaces

- Embora a definição de interfaces seja útil, ela cria uma rigidez no projeto de sistemas:
 - SO só irá funcionar sobre o hardware (ISA) para qual foi construído
 - Bibliotecas só irão funcionar sobre a ABI para a qual foi projetada
 - Aplicações devem obedecer as ABIs e APIs pré-definidas
- Interação inflexível entre interfaces de componentes de um sistema operacional traz alguns problemas
 - **Baixa portabilidade:** sem mobilidade de código e interoperabilidade, acoplando de forma excessiva aplicações ao SO + hardware
 - **Barreiras de inovação:** interfaces rígidas dificultam interações entre aplicações e dispositivos de hardware, inércia à evolução
 - **Otimizações intercomponentes:** otimizações são feitas apenas a nível de cada componente (abstração)

Virtualização de interfaces

- Contornar problemas relacionados à inflexibilidade de interfaces através de **camada de virtualização** em software
 - Essencialmente permitindo que um programa desenvolvido para uma plataforma possa executar em outra
- Camada de virtualização utiliza serviços de uma interface do sistema para **oferecer outra interface de mesmo nível**
 - Nova interface de sistema fornecida através da camada de virtualização é chamada **máquina virtual**
 - Camada de virtualização em si é chamada de **hypervisor** ou **monitor de máquina virtual (VMM)**

Virtualização de interfaces

- Um ambiente de máquina virtual consiste em três partes:
 - **Sistema real, nativo ou hospedeiro** (*host system*), contendo os recursos reais do sistema
 - **Camada de virtualização**, chamada de *hypervisor* ou VMM, que fornece uma interface virtual
 - **Sistema virtual ou convidado** (*guest system*), executando sobre a camada de virtualização

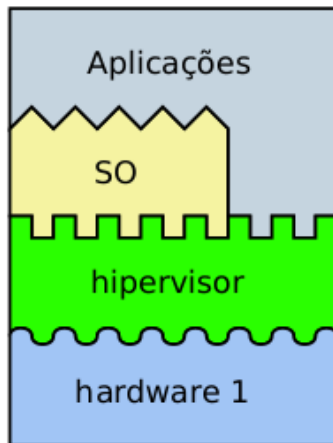
Virtualização de interfaces

- Tipos de virtualização
 - **Emulação de hardware:** sistema operacional convidado e suas aplicações desenvolvidas para uma plataforma são executadas sobre outra plataforma distinta
 - **Emulação de sistema operacional:** aplicações construídas para um sistema operacional são executadas sobre outro sistema operacional
 - **Otimização dinâmica:** instruções de máquina das aplicações são traduzidas durante a execução em outras instruções mais eficientes para a mesma plataforma
 - **Replicação de hardware:** várias instâncias virtuais de um mesmo hardware real, cada uma executando seu próprio sistema operacional convidado e suas respectivas aplicações

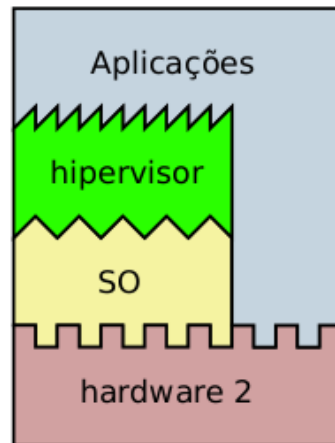
** Emulação é uma forma de virtualização onde hypervisor virtualiza integralmente uma interface de hardware ou SO*

Virtualização de interfaces

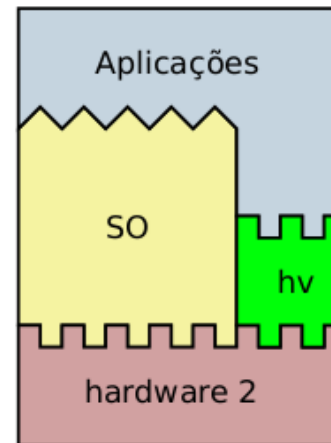
- Tipos de virtualização



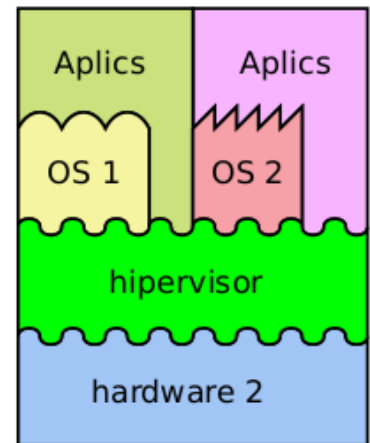
emulação
de hardware



emulação do SO



otimização dinâmica

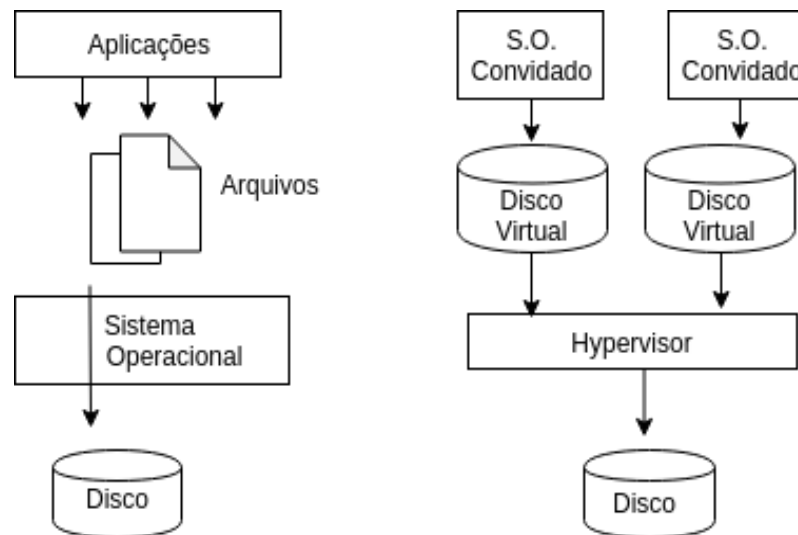


replicação
de hardware

Fonte: Maziero, 2019

Diferença entre virtualização e abstração

- Objetivos diferentes
 - Sistema operacional fornece visão alto nível de recursos de hardware em níveis de abstração crescentes
 - Virtualização cria novas interfaces a partir das existentes
 - Detalhes de baixo nível da plataforma real não são necessariamente ocultos, como ocorre na abstração de recursos

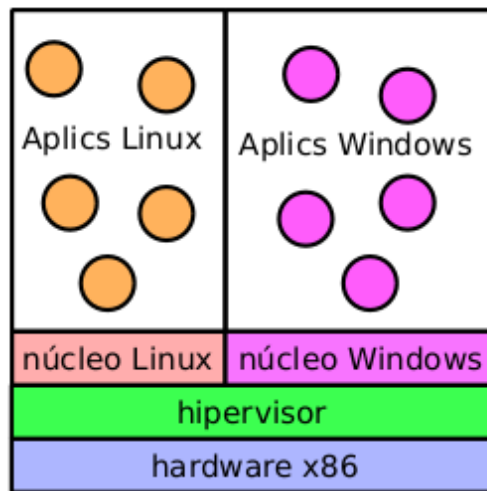


Tipos de máquinas virtuais

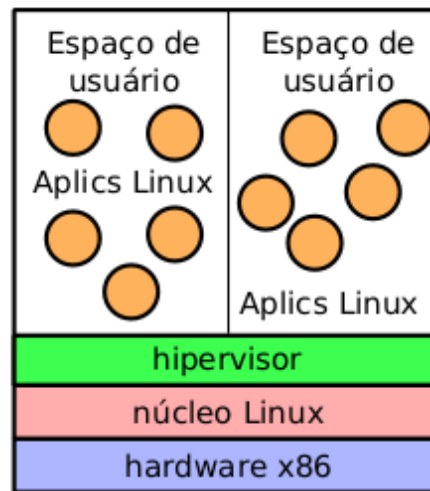
- Máquinas virtuais podem ser classificadas de acordo com o ambiente virtual fornecido
 - **Máquinas virtuais de sistema**
 - Construídas para emular plataformas de hardware completas
 - Suportam sistemas operacionais executando aplicações
 - Ex: KVM, VMware e VirtualBox
 - **Máquinas virtuais de sistema operacional**
 - Suportam espaços de usuário distintos sobre um mesmo SO
 - Possuem recursos lógicos próprios (armazenamento, interfaces de rede, ...)
 - Ex: Docker, Solaris Containers, ...
 - **Máquinas virtuais de processo**
 - Ou máquinas virtuais de aplicação ou linguagem
 - Suportam a execução de apenas um processo ou aplicação convidada específica
 - Ex: Java, Valgrind

Tipos de máquinas virtuais

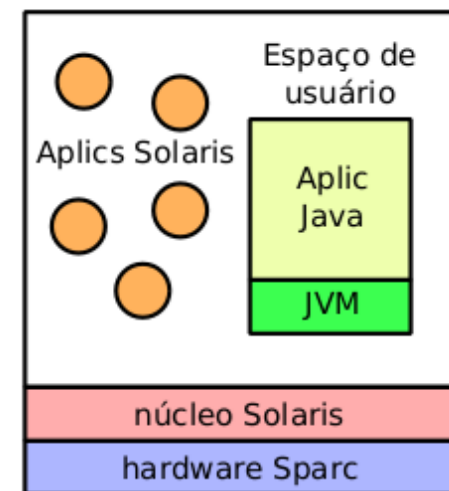
- Máquinas virtuais podem ser classificadas de acordo com o ambiente virtual fornecido



VM de sistema
(hardware)



VM de sistema
operacional



VM de processo

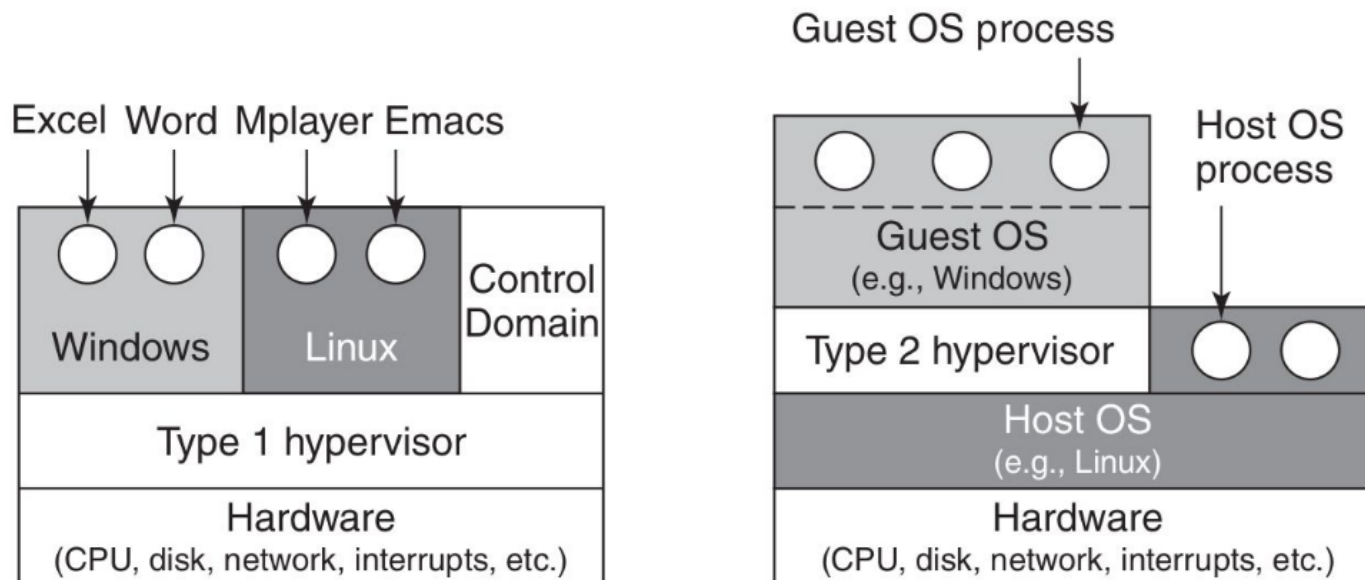
Fonte: Maziero, 2019

Tipos de máquinas virtuais

- **Máquinas virtuais de sistema**
 - Cada sistema tem a ilusão de executar sozinho sobre uma plataforma de hardware exclusiva
 - Hypervisor intercepta e virtualiza o acesso aos recursos
 - Forte isolamento entre máquinas virtuais em uma mesma plataforma
 - Existem duas arquiteturas para hypervisors
 - **Hypervisor Nativo (tipo I)**
 - Executa diretamente sobre hardware, sobre nenhum sistema operacional
 - Cada máquina virtual se comporta como um computador completo
 - Ex: VMware ESX Server e Xen
 - **Hypervisor Convidado (tipo II)**
 - Executa como processo de um SO subjacente, que gerencia o hardware
 - Hypervisor oferece recursos virtuais aos SOs convidados
 - Ex: VMware Workstation e VirtualBox

Tipos de máquinas virtuais

- Máquinas virtuais de sistema



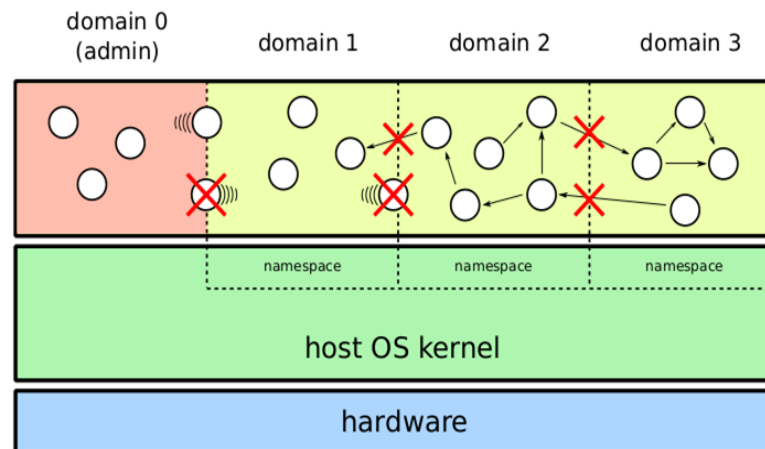
Hypervisors tipo I e tipo II. Fonte: Tanenbaum (2015)

Tipos de máquinas virtuais

- **Máquinas virtuais de sistema operacional**
 - Custo da virtualização de plataforma de hardware sobre o desempenho do sistema pode ser elevado
 - Virtualização dos recursos (periféricos) da máquina real
 - Possível necessidade de emular instruções do processador real
 - Virtualização do espaço de usuário (*userspace*) provê alternativa para implementar **isolamento entre aplicações** ou subsistemas
 - Espaço de usuário do SO é dividido em áreas isoladas denominadas **domínios** ou **contêineres**
 - Parcela dos recursos do sistema operacional alocada a cada domínio
 - Núcleo do sistema é o mesmo para todos os domínios
 - Conjunto de chamadas de sistema é preservado
 - Ex. LXC e Docker

Tipos de máquinas virtuais

- **Máquinas virtuais de sistema operacional**
 - Processos de um domínio podem interagir entre si, mas não com processos de outro domínio (a não ser através da rede)
 - Normalmente processos no domínio d0, chamado de domínio inicial, privilegiado ou de gerência
 - Têm visibilidade e acesso aos processos e recursos dos demais domínios



Fonte: Maziero, 2019

Tipos de máquinas virtuais

- **Máquinas virtuais de processo**
 - Suporta execução de um processo ou aplicação individual
 - Criada e destruída para execução de uma aplicação
 - Hypervisor + aplicação visto como único processo dentro do SO
 - Hypervisor pode implementar mesma interface de hardware (ISA) da máquina real subjacente, ou interface distinta
 - Exemplos de máquinas virtuais de aplicação
 - Sistemas operacionais multitarefa (PCB + registradores!)
 - Tradutores dinâmicos, alguns depuradores de memória
 - Mapeamento de chamadas de sistema e de bibliotecas usadas pela aplicação para chamadas equivalentes oferecidas pelo SO da máquina real
 - Ambiente Wine: permite executar aplicações Windows em plataformas Unix
 - Mapeamento interface ISA abstrata para interface ISA de hardware real
 - Compilação de programa Java gera bytecodes para máquina abstrata

Requisitos básicos para hypervisors

- Monitor ou hypervisor deve atender a alguns requisitos básicos [Popek and Goldberg, 1974]
 - **Equivalência**
 - Prover ambiente idêntico à máquina real, aplicações executam com mesmo comportamento que teriam em máquina real
 - **Controle de recursos**
 - Hypervisor possui controle completo dos recursos na máquina real e intermedia acesso a eles pela VM
 - **Eficiência**
 - Grande parte das instruções do processador virtual devem ser executadas pelo processador real, sem intervenção do hypervisor

Requisitos complementares para hypervisors

- Propriedades adicionais frequentemente associadas a hypervisors [Rosenblum, 2004]
 - **Isolamento**
 - Sem interação direta das aplicações de uma VM com outras aplicações, com o hypervisor ou com o sistema hospedeiro
 - **Introspecção**
 - Hypervisor tem acesso e controle sobre o estado da VM, incluindo registradores, conteúdo de memória, ...
 - **Recursividade**
 - Pode ser possível executar um hypervisor dentro de uma máquina virtualizada

Construção de máquinas virtuais

- No entanto, hypervisor ideal nem sempre pode ser construído sobre plataformas de hardware existentes
- Possibilidade de construção de hypervisor em determinada plataforma definida pelo **teorema de Popek e Goldberg [1974]**
 - “Para qualquer computador convencional de terceira geração, um hypervisor pode ser construído se o conjunto de instruções sensíveis daquele computador for um subconjunto de seu conjunto de instruções privilegiadas.”
- Definições
 - **Instruções sensíveis:** podem consultar ou alterar estado do processador, p.ex., registradores que armazenam status da execução
 - **Instruções privilegiadas:** acessíveis somente por código em modo privilegiado
 - Quando executadas por código não privilegiado devem gerar uma interrupção a ser tratada

Construção de máquinas virtuais

- Implicações do teorema de Popek e Goldberg
 - Toda instrução sensível deve ser também privilegiada
 - Quando instrução sensível é executada por programa não privilegiado (p.ex. kernel convidado), provocará uma interrupção
 - Interrupção pode ser usada para ativar rotina de interpretação dentro do hypervisor, que irá simular o efeito da instrução sensível
 - Maior número de instruções sensíveis, leva a maior o volume de interpretação de código
 - Pior será o desempenho da máquina virtual
- Alternativa em sistemas que não satisfazem o teorema
 - Paravirtualização: reescrever parte do sistema convidado para não usar essas instruções sensíveis

Suporte de hardware

- Suporte de HW para hypervisors deficiente por muito tempo
 - Intel Pentium IV (e anteriores) possuía 17 instruções sensíveis que podiam ser executadas em modo usuário (violando teorema)
 - **SGDT/SLDT**: ler registrador com posição/tamanho de tabelas de segmentos
 - **SMSW**: ler registrador com status do processador
 - **PUSHF/POPF**: empilha/desempilha valor do registrador com status do processador
 - Técnicas iniciais de virtualização para Intel/AMD se baseavam em redução de privilégios do SO convidado (anéis de proteção)
 - Hypervisor e SO hospedeiro executam no nível 0
 - SO convidado executa no nível 1 ou 2
 - Aplicações do sistema convidado executam no nível 3
 - Algumas instruções do SO convidado precisavam ser reescritas (**paravirtualização!**) dinamicamente no momento da carga
 - Pois SO foi originalmente construído para executar no nível 0

Suporte de hardware

- Por volta de 2005, fabricantes de microprocessadores (Intel e AMD) incorporaram suporte básico à virtualização
 - IVT (Intel Virtualization Technology) e AMD-V (AMD Virtualization)
- Ideia central de ambas as tecnologias
 - Definir dois modos possíveis de operação do processador
 - **Modo root:** equivale ao funcionamento de processador convencional, destinado à execução de um hypervisor
 - **Modo non-root:** se destina à execução de máquinas virtuais
 - Ambos os modos suportam os quatro níveis de privilégio
 - Permite executar SOs convidados sem necessidade de reescrita dinâmica de código

Níveis de virtualização

- Virtualização implica na reescrita de interfaces de sistema e de acordo com a interface, há diferentes níveis de virtualização
 - **Virtualização do hardware:** virtualiza toda a interface ISA, que dá acesso ao hardware
 - **Virtualização da interface de sistema:** virtualiza a System ISA, que corresponde ao conjunto de instruções sensíveis do processador
 - **Virtualização de dispositivos de I/O:** virtualiza dispositivos físicos, como discos e interfaces de rede
 - **Virtualização do sistema operacional:** cada VM é como uma instância distinta do mesmo SO subjacente
 - **Virtualização de chamadas de sistema:** virtualiza chamadas de sistema de um sistema A para um sistema B
 - **Virtualização de chamadas de biblioteca:** virtualiza chamadas de biblioteca

Técnicas de virtualização

- Estratégias mais utilizadas atualmente para virtualização:
 - Emulação completa do hardware
 - Virtualização da interface de sistema
 - Tradução dinâmica de código
 - Paravirtualização

Técnicas de virtualização

- **Emulação completa do hardware**
 - Virtualiza toda a interface de hardware, podendo oferecer ao sistema convidado uma interface distinta da física
 - Incluindo instruções do processador, a memória e dispositivos
 - Realiza tradução dinâmica das instruções
 - Exemplo: VMs QEMU (oferecem processador x86 a sistema convidado)
- **Virtualização da interface de sistema**
 - Interface ISA de usuário é mantida, e apenas instruções privilegiadas e dispositivos (discos, interfaces de rede, etc.) são virtualizados
 - Instruções sensíveis devem gerar interrupções
 - Sistema convidado e aplicações enxergam o processador real
 - Melhor desempenho do SO convidado, próximo à execução em hardware real
 - Exemplos: VMware Workstation, VirtualBox e KVM

Técnicas de virtualização

- **Tradução dinâmica**

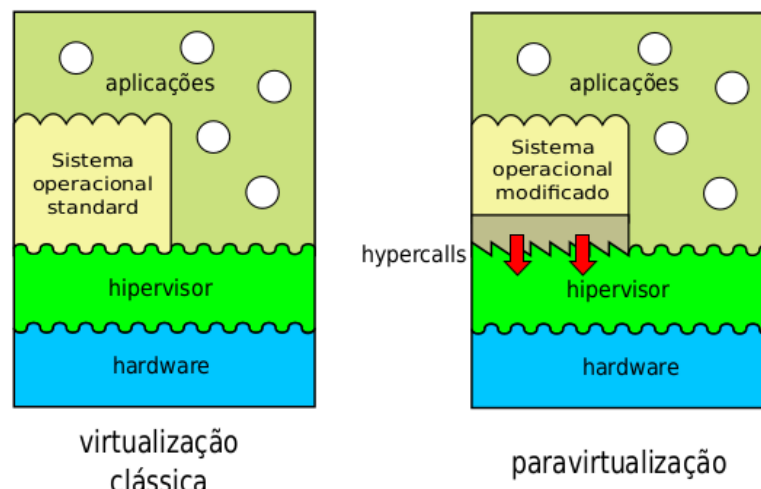
- Hypervisor analisa, reorganiza e traduz sequências de instruções realizadas pelo sistema convidado
- Conforme código é carregado em memória ou durante a execução
- Possíveis objetivos:
 - Adaptar instruções geradas pelo sistema convidado à interface ISA do sistema real
 - Detectar e analisar instruções sensíveis não-privilegiadas
 - Melhorar o desempenho da execução das instruções através de otimizações
- Exemplo: bytecodes Java executando em JVM são interpretados e traduzidos para interface do sistema real

Técnicas de virtualização

Fonte: Maziero, 2019

- **Paravirtualização**

- Modifica interface entre hypervisor e SOs convidados para fornecer hardware virtual similar ao real
- Hypervisor define API de chamadas de hypervisor (*hypercalls*) para que cada convidado acesse a interface de sistema de hardware virtual
 - Modificações na interface de sistema do hardware virtual (system ISA) exigem adaptação dos sistemas operacionais convidados
- Interface de usuário (user ISA) do hardware é preservada
 - Permite que aplicações convidadas executem sem necessidade de modificações
- Exemplo: Xen permite SOs convencionais como Linux e Windows (modificados) executar sobre um hypervisor



Migração de máquinas virtuais

- Permite transferir VM e seu sistema convidado através da rede de um hypervisor para outro, executando em equipamentos distintos, sem necessidade de reiniciá-la
 - **Stop-and-copy:** suspende a VM e transfere o *conteúdo de sua memória* para o hypervisor de destino e depois retoma sua execução
 - Simples, porém implica em parar completamente os serviços
 - **Demand-migration:** VM suspensa apenas durante a cópia das *estruturas de memória* do SO convidado para hypervisor de destino
 - Após, máquina virtual é retomada e restante das páginas de memória é transferido sob demanda
 - **Pre-copy:** copia para o hypervisor destino todas as páginas de memória da máquina virtual enquanto ela executa
 - VM é suspensa e as páginas modificadas durante o processo são novamente copiadas para o destino

Virtualização na prática

- Aplicações da virtualização
 - Construção de binários portáveis
 - Compartilhamento de hardware
 - Suportar aplicações legadas
 - Experimentação em redes
 - Ensino
 - Segurança
 - Desenvolvimento de baixo nível
 - Tolerância a falhas
 - Computação sob demanda
 - Virtualização de redes

Referências

- Maziero, Carlos – “Sistemas Operacionais: Conceitos e Mecanismos”, 2019
(<http://wiki.inf.ufpr.br/maziero/doku.php?id=socm:start>)
 - Capítulo 31
 - Capítulo 32
 - Capítulo 33
 - Capítulo 34
- Tanenbaum, A. Boss, H. – “Modern Operating Systems” (4th edition), Pearson, 2015.
 - Capítulo 7